ERP PURCHASE, QUALITY CONTROL & STOCK MANAGEMENT

Project documentation • Windows/XAMPP setup • Demo credentials • Functional testing guide

Project: erp_complete Stack: Laravel 13, Vue 3, MySQL, Vite Document date: 11 November 2026

Scope note: This guide is based on the ERP project source ZIP and its README. The user confirmed that setup is working. The complete business workflow has not been independently executed or verified in this environment.

1. Project overview

The application demonstrates a purchase-to-invoice workflow with item-level QC configuration, approved purchase orders, goods receipt lots, sample measurements, QC authorization, stock release, and supplier invoicing.

Business flow: Item Master → Purchase Order → PO Approval → Goods Receipt (GRN) → Lot QC Inspection → QC Approval → Stock Movement → Supplier Invoice.

2. Technology stack

Layer	Technology / purpose
Backend	PHP with Laravel 13; Eloquent ORM; controllers and session authentication
Frontend	Vue 3 single-page interface; Vite build/development tooling
Database	MySQL (via XAMPP); Laravel migrations and seeders
Runtime & tooling	Composer, Node.js, npm, PowerShell/CMD
Local web access	Laravel development server at http://127.0.0.1:8000/erp

3. Main modules

Module	Capabilities
Item Master	Item code, category, UOM, QC-required flag, parameter limits, lot sizing, sampling rules
Purchase Orders	Supplier, warehouse, line quantities, prices, tax, submit and manager approval
GRN / Lots	Approved PO linkage, partial receipt, multiple lots, hold inventory
QC Inspections	Per-lot sample count, required measurements, limits snapshot, submit and approve
Stock Management	Hold → available for passing lots; hold → rejected for failed lots; movement ledger
Supplier Invoices	Accepted-lot quantities, partial invoicing, supplier invoice uniqueness, tax totals
Audit trail	Audit log records for significant operations

4. Installation from scratch — Windows + XAMPP

Prerequisites: PHP 8.3+, Composer, Node.js 20.19+ (or 22.12+), npm, XAMPP with MySQL. Start MySQL from the XAMPP control panel. Verify tool versions:

```
php -v
composer -V
node -v
npm -v
```

Create a fresh Laravel project in your actual installation directory (example below uses E:).

```
cd E:\xampp\htdocs
composer create-project laravel/laravel erp_complete "^13.0"
```

Extract erp_purchase_qc_complete.zip and copy its contents into E:\xampp\htdocs\erp_complete, replacing matching starter files. The ZIP is a source overlay; vendor/ and node_modules/ are not bundled.

Create a database named erp_complete in phpMyAdmin, then configure the project .env:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=erp_complete
DB_USERNAME=root
DB_PASSWORD=
```

Keep the existing APP_KEY; if missing, run php artisan key:generate. Set APP_URL to http://127.0.0.1:8000.

```
cd E:\xampp\htdocs\erp_complete
composer install
npm install
```

```
npm install vue
```

```
npm install -D @vitejs/plugin-vue
```

```
php artisan migrate --seed
```

Known migration fix (MySQL error 1059)

In database/migrations/2026_01_01_000001_create_erp_tables.php, the automatically generated unique index for qc_inspection_results exceeds the MySQL identifier length limit. Replace the unique declaration with a short explicit name:

```
$t->unique(['qc_inspection_sample_id','parameter_name'], 'qc_result_sample_param_unique');
```

For a new disposable test database only, rerun php artisan migrate:fresh --seed after the edit. WARNING: migrate:fresh drops all tables and data in the configured database.

Start the application

Terminal A:

```
php artisan serve
```

Terminal B:

```
npm run dev
```

Open http://127.0.0.1:8000/erp. Use Laravel port 8000, not the Vite development port. If you get 404, check php artisan route:list --path=erp and php artisan optimize:clear.

5. Demo login credentials (local development only)

Role	Email	Password
Buyer	buyer@example.test	password
Manager	manager@example.test	password
QC Inspector	inspector@example.test	password
QC Approver	qc_manager@example.test	password
Accounts	accounts@example.test	password

Correction: The seeded QC approver account is qc_manager@example.test, because the seeder constructs email addresses directly from role names. If you previously tried qcmanager@example.test, that address is not created by this seeder. All accounts use the demonstration password password. Never use these credentials in production.

6. Seeded test data

Record	Value
Supplier	Demo Steel Supplier
Warehouse	WH1 — Main Warehouse
Item	ITEM001 — Steel Component; category Raw Material; UOM PCS
QC limits	Height, Width, Length, Thickness: 1–20 mm; Weight: 1–20 g
Lot size	Minimum 1; maximum 5000 PCS

QC sampling rules

Lot quantity	Required samples
1–100	2 (capped at lot quantity)
101–500	5
501–1000	10
1001+	15

7. End-to-end functional test procedure

Suggested scenario: Order and receive 300 PCS of ITEM001 in one lot. Use the seeded supplier and warehouse.

• Test 1 — Login: Sign in as buyer@example.test; confirm the dashboard loads and seeded master data is visible.

• Test 2 — Purchase Order: As Buyer, create a PO for ITEM001, 300 PCS, with a unit price and tax rate. Submit it. Confirm the PO is recorded.

• Test 3 — Approval: Sign out and log in as Manager. Approve the submitted PO. Confirm an unapproved PO cannot be received.

• Test 4 — Goods Receipt: Log in as Buyer. Receive 300 PCS against the approved PO in lot LOT-001. Confirm the lot is on QC hold and overreceipt is rejected.

• Test 5 — QC creation: Log in as QC Inspector. Start an inspection for LOT-001. A 300-unit lot should generate five samples.

• Test 6 — QC pass: Enter all five measurements for each of five samples (25 readings total), each within its configured 1–20 limit. Submit the inspection.

• Test 7 — QC approval: Log in as QC Approver (qc_manager@example.test). Approve the submitted inspection. Confirm 300 PCS move from hold to available exactly once.

• Test 8 — Invoice: Log in as Accounts. Create an invoice against accepted quantity. Confirm totals include tax and accepted quantity cannot be billed twice.

8. Negative and boundary test cases

Case	Expected behavior
Lot quantity = 1	1 sample, despite rule specifying 2
Lot quantity = 80	2 samples
Lot quantity = 300	5 samples
Lot quantity = 800	10 samples
Lot quantity = 1500	15 samples
Measured value = 20	Pass when configured maximum is 20
Measured value = 21	Fail lot when required parameter maximum is 20
Missing sample reading	Inspection submission blocked
Receipt exceeds PO balance	GRN creation rejected
Unauthorized QC approval	Approval rejected
Repeated QC approval	No duplicate stock release
Rejected lot invoicing	Rejected quantity not eligible as accepted stock

9. API reference (implemented routes)

These routes are declared in routes/web.php under the /api prefix. Authentication uses the Laravel web session and CSRF protection for write requests; the application is not configured as a public token API.

Method	Route
POST	/api/login
POST	/api/logout
GET	/api/me
GET	/api/overview
POST	/api/items

Method	Route
GET	/api/items/{item}/qc-samples
POST	/api/suppliers
POST	/api/warehouses
POST	/api/purchase-orders
POST	/api/purchase-orders/{po}/submit
POST	/api/purchase-orders/{po}/approve
POST	/api/goods-receipts
POST	/api/lots/{lot}/inspect
POST	/api/qc-inspections/{inspection}/measurements
POST	/api/qc-inspections/{inspection}/submit
POST	/api/qc-inspections/{inspection}/approve
POST	/api/supplier-invoices

10. Troubleshooting

• 404 on /erp: Check the URL/port, route:list, route cache, and resources/views/erp.blade.php.

• Vite manifest error: Run npm install and npm run dev; check vite.config.js and resources/js/app.js.

• Database connection error: Start MySQL in XAMPP; confirm DB_DATABASE, credentials, and port in .env.

• MySQL error 1059: Use the short explicit unique index name documented in section 4.

• 401/419 errors: Log in through /erp and ensure session cookies and CSRF token are sent on writes.

• Changes not appearing: Run php artisan optimize:clear and restart the relevant development server.

11. Limitations and verification status

This is an assessment/demo source package. The provided README states that supplier returns, debit notes, and credit notes are not implemented. Inventory is derived from stock movements rather than a cached balance table. Production hardening, deployment security, backups, and a complete automated test suite remain outside the supplied package.

Verification status: The user reported successful setup and a working application after fixing the migration constraint. The end-to-end functional and negative tests in this document are test instructions and expected outcomes, not claims that all scenarios have passed.

Source: erp_purchase_qc_complete.zip (README.md, routes/web.php, database/seeders/DatabaseSeeder.php, database/migrations/2026_01_01_000001_create_erp_tables.php).